Fitness Tracker

Landon Duba, Sid Madireddy, Justin Hwang, Vihaan Jaiswal

Point of contact email addresses: sidm@vt.edu, landonduba@vt.edu, justinjhwang@vt.edu, vihaan@vt.edu

CS 4604, Dr. Nizamani

Spring 2026

Table of Contents

1. Introduction to the Mini-verse 3
1.1 Background 3
1.2 Motivation 3
1.3 Overview of the Mini-verse 3
2. User Support 4
2.1 User Roles and Entities 4
2.2 Authentication (Login System) 4
2.3 Security Measures 4
3. Database Connection & Setup 5
3.1 System Connection 5
3.2 Setup Instructions 5
3.3 Database Initialization 5
3.4 Challenges & Lessons Learned 5
4. System Design & Functionality 6
4.1 Schema Design 6
4.2 ER Diagram 6
4.3 Tables & Keys 6
5. Reporting 7
6. GUI / Interface 8
6.1 CRUD Operations 8
7. Reflection 9
7.1 Individual Contributions 9
7.2 Team Reflection 9
8. Appendix 10

1. Introduction to the Mini-verse

This section introduces your project, its purpose, and the overall system you have designed.

1.1 Background

Many gym-goers struggle to track their workouts and nutrition consistently. Most people rely on notes apps, spreadsheets, or memory, which leads to missing entries, duplicate data, and no easy way to look back at progress. Our system solves this by providing a centralized, database-driven platform where users can log and manage all their fitness and diet data in one place.

1.2 Motivation

We built this for anyone trying to improve or maintain their health, from beginners just starting out to advanced athletes tracking detailed performance metrics. The core actions users perform are logging meals, recording workouts, and tracking body metrics over time. Without a structured system, it's nearly impossible to spot trends or make informed adjustments to training and diet.

1.3 Overview of the Mini-verse

- The system is organized around eight primary entities stored in a MySQL relational database:
- • USER — stores account credentials, user type (regular or admin), and creation timestamp.
- • MEAL — represents a logged meal session tied to a specific user and datetime.
- • FOOD — a shared catalog of food items with nutritional data per serving (calories, protein, carbs, fat, fiber).
- • MEAL_FOOD — junction table resolving the many-to-many relationship between meals and food items, storing quantity in servings.
- • WORKOUT — a logged workout session tied to a user, with total duration and calories burned.
- • EXERCISE — a shared catalog of exercises with associated muscle group and calorie estimate.

- • WORKOUT_EXERCISE — junction table resolving the many-to-many relationship between workouts and exercises, storing sets, reps, weight, duration, and exercise order.
- • PROGRESS — a daily snapshot of a user's body weight and body fat percentage, with a unique constraint on (user_id, progress_date) to enforce one entry per day.

- Users interact with the system through a web-based GUI built with React and plain CSS, which communicates with a Python Flask backend. The Flask server handles all SQL queries against the MySQL database, ensuring users never interact with the database directly. The key relationships in the system are: a user has many meals and many workouts; a meal contains many foods and a food can appear in many meals; a workout includes many exercises and an exercise can appear in many workouts; and a user has many progress records, one per calendar day.

2. User Support

The system defines two user roles, differentiated by the user_type attribute on the USER entity:

2.1 User Roles and Entities

Role 1: General User (Regular Gym-Goer)

The General User is the primary persona for the application. This user has beginner-to-intermediate gym experience and wants a simple, intuitive way to manage health data. Their permitted actions include:

- Create, read, update, and delete their own workout logs.
- Create, read, update, and delete their own meal logs.
- Browse the food and exercise catalogs.
- View personal fitness history and daily/weekly summaries.
- Track and update their own progress records (body weight, body fat).
- Manage their personal profile.

Role 2: Admin

The Admin is responsible for system-level management and does not participate in personal fitness tracking. Admin permissions include:

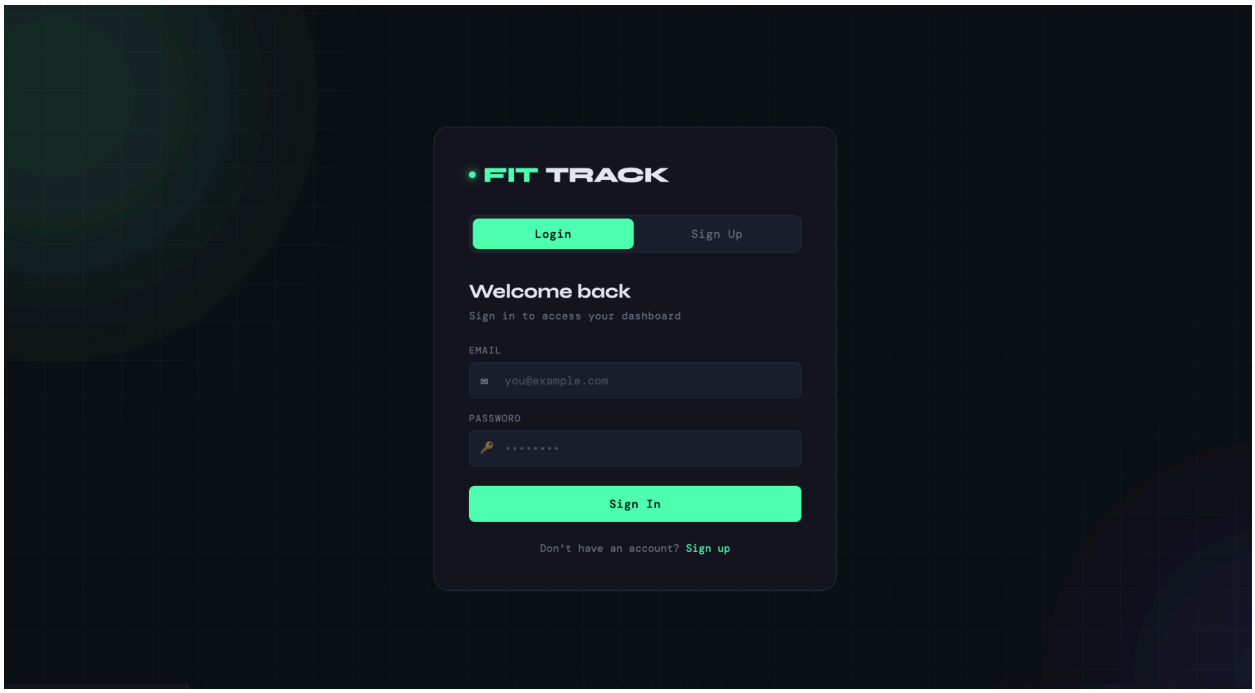
- Approve, deactivate, or delete user accounts.

- Create, update, and delete entries in the shared FOOD and EXERCISE catalogs.
- Monitor system activity and generate aggregate usage reports.

2.2 Authentication (Login System)

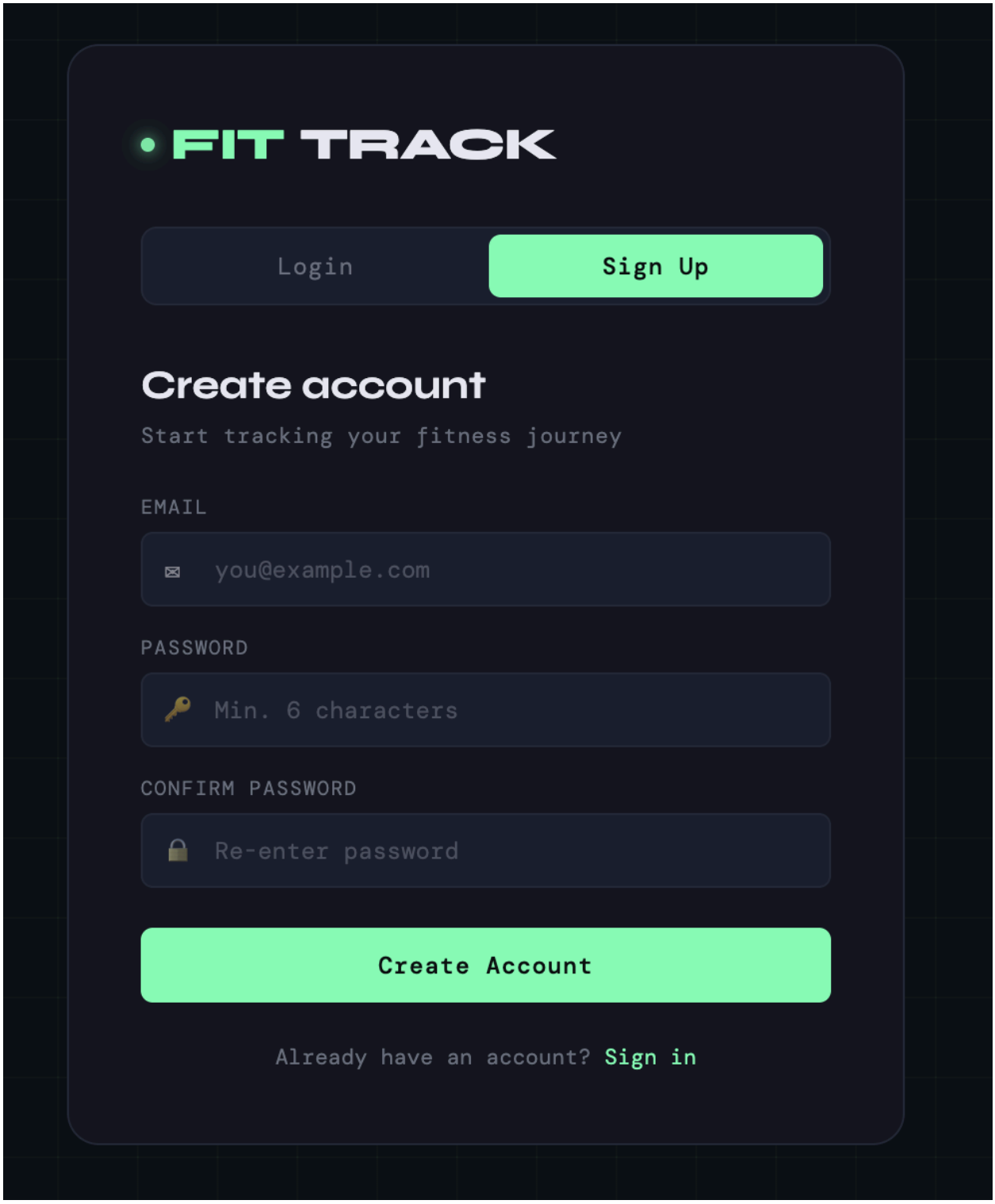
Include screenshots that demonstrate:

- successful login/logout:



[1] - This image shows the screen after a logout. After someone logs out they are sent back to the login/signup page.

- account creation or update



[2] - The account creation page is featured here under the ‘sign up’ tab.

2.3 Security Measures

SQL injection is prevented by using parameterized queries throughout the Flask backend, user input is never concatenated directly into SQL strings. Passwords are hashed before storage, so the database never holds plaintext credentials. Flask sessions are used to manage authentication state server-side, and sensitive data is never exposed in URLs.

Include supporting screenshots or code snippets where appropriate.

3. Database Connection & Setup

This section explains how your system connects to the database and how it can be set up and reproduced.

3.1 System Connection

- The app follows a three-tier architecture: a React/CSS frontend sends HTTP requests to a Python Flask backend, which executes SQL queries against a MySQL database. Flask connects to MySQL using a Python database connector library. The frontend queries the backend with endpoints, which, when needed, query the database.

```python
app = Flask(__name__)
app.secret_key = "change-this-to-a-random-secret"
app.config.update(SESSION_COOKIE_SAMESITE="Lax")

CORS(app, supports_credentials=True, origins=["http://127.0.0.1:5500"])

ADMIN_CODE = "fittrack-admin-2024"

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "password",
    "database": "fitness_db",
}

def get_db():
    return mysql.connector.connect(**DB_CONFIG)

def query(sql, params=(), fetchone=False, commit=False):
    conn = get_db()
    cur = conn.cursor(dictionary=True)
    cur.execute(sql, params)
    if commit:
        conn.commit()
        last_id = cur.lastrowid
        cur.close(); conn.close()
        return last_id
    result = cur.fetchone() if fetchone else cur.fetchall()
    cur.close(); conn.close()
    return result
```

[3] - This code snippet displays our database connection setup

3.2 Setup Instructions

To replicate our setup someone would need access to React and Python (with Flask). In order to use react they can use the node package manager to get the dependencies or pull the packages from a CDN like cloudflare. They can download python from online and use 'pip install flask' to download flask. For password hashing they can similarly do 'pip install bcrypt', the encryption package that we used. For the database they would need access to MySQL, which can again be downloaded online. Then to run the project, they could use a live server extension in their code editor, or they could use 'npm start dev'. We used react by pulling it from a CDN so all the react lives in an html file, so we used the live server extensions throughout our development. Before starting the frontend, they can simply run the backend app.py file to get the server running, which has the password needed for the database.

3.3 Database Initialization

- Tables are created via a schema.sql file that defines all eight tables in dependency order, starting with USER and ending with the junction tables. Sample data is loaded via a seed.sql file. Major tables like USER, MEAL, WORKOUT, and PROGRESS each have at least 20 records. Supporting tables like FOOD and EXERCISE have fewer records proportional to their role.

3.4 Challenges & Lessons Learned

- The biggest early challenge was correctly modeling the many-to-many relationships. The Phase 2 ERD didn't clearly define junction tables, which caused cardinality confusion. This was fixed in Phase 3 by introducing MEAL_FOOD and WORKOUT_EXERCISE with composite primary keys. Another issue was allowing the same exercise to appear more than once in a single workout, solved by adding exercise_order to the composite key of WORKOUT_EXERCISE. We also learned to define constraints at the database level rather than relying only on the application layer.

3.5 Sample Data:

Here is an example of sample data from our exercises table:

Exercises

Exercise library with muscle groups

Search exercises...

EXERCISE	MUSCLE GROUP	CAL/UNIT
Arnold Press	Shoulders	7
Barbell Curl	Arms	5
Barbell Row	Back	9
Battle Ropes	Full Body	10
Bench Press	Chest	8
Burpee	Cardio	10
Cable Crunch	Core	4
Cable Fly	Chest	6
Cable Kickback	Glutes	4
Clean and Press	Full Body	11
Crunch	Core	3
Cycling	Cardio	8
Deadlift	Back	12
Dips	Chest	7
Face Pull	Shoulders	5

[4] - An example of some sample data

3.6 Example Queries:

We performed the queries using the following query function:

```
def query(sql, params=(), fetchone=False, commit=False):

   conn = get_db()

   cur = conn.cursor(dictionary=True)

   cur.execute(sql, params)

   if commit:

       conn.commit()

       last_id = cur.lastrowid

       cur.close(); conn.close()

       return last_id

   result = cur.fetchone() if fetchone else cur.fetchall()

   cur.close(); conn.close()

   return result
```

[5] - Code snippet of setting up the query function, allows for easier functions on the backend

Then, when we wanted to make a query we could simple do something similar to the following examples:

```
user_id = query(

      "INSERT INTO USER (email, password_hash, user_type, created_at) VALUES
(%s,%s,%s,%s)",

       (email, pw_hash, user_type, date.today().isoformat()), commit=True

   )
```

```
last_id = query(

      "INSERT INTO EXERCISE (exercise_name, muscle_group, calories_per_unit) VALUES
(%s,%s,%s)",

       (d["exercise_name"], d.get("muscle_group"), d.get("calories_per_unit")),
commit=True

   )
```

[6] - Example queries from the codebase

4. System Design & Functionality

This section describes your database design and how it supports your application.

4.1 Schema Design

The schema for the Fitness and Diet Tracker evolved from an initial conceptual ER design into a fully normalized relational schema.

In the earlier phase, the design did not clearly resolve many-to-many relationships, which led to ambiguity in how meals and workouts were connected to food and exercise. To fix this, we introduced two junction tables: **MEAL_FOOD** and **WORKOUT_EXERCISE**. These tables allow a single meal to contain multiple food items and a single workout to include multiple exercises, while also allowing reuse of those items across different records.

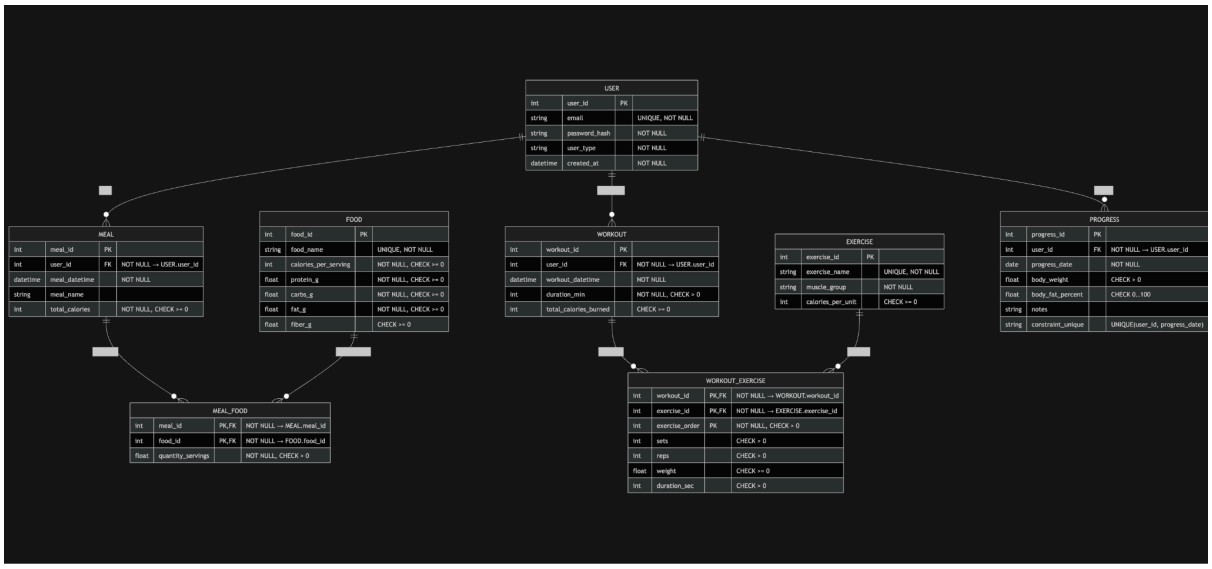
Additionally, the **WORKOUT_EXERCISE** table was enhanced by adding the `exercise_order` attribute to allow the same exercise to appear multiple times within a single workout session. This ensures flexibility and supports more realistic workout tracking.

The schema follows **Third Normal Form (3NF)**:

- Each table represents a single entity.
- All non-key attributes depend only on the primary key.
- There are no transitive dependencies.

This normalization reduces redundancy, improves data consistency, and ensures efficient querying across the system.

4.2 ER Diagram



[7] - The ER Diagram we created for our project

4.3 Tables & Keys

USER
- PK: user_id
- email (UNIQUE, NOT NULL)
- password_hash (NOT NULL)
- user_type (NOT NULL)
- created_at (NOT NULL)

MEAL
- PK: meal_id
- FK: user_id → USER.user_id (NOT NULL)
- meal_datetime (NOT NULL)
- meal_name
- total_calories (NOT NULL, CHECK total_calories >= 0)

FOOD
- PK: food_id
- food_name (UNIQUE, NOT NULL)
- calories_per_serving (NOT NULL, CHECK >= 0)
- protein_g (NOT NULL, CHECK >= 0)
- carbs_g (NOT NULL, CHECK >= 0)
- fat_g (NOT NULL, CHECK >= 0)
- fiber_g (CHECK >= 0)

MEAL_FOOD
- PK: (meal_id, food_id)
- FK: meal_id → MEAL.meal_id (NOT NULL)
- FK: food_id → FOOD.food_id (NOT NULL)
- quantity_servings (NOT NULL, CHECK > 0)

WORKOUT
- PK: workout_id
- FK: user_id → USER.user_id (NOT NULL)
- workout_datetime (NOT NULL)
- duration_min (NOT NULL, CHECK > 0)
- total_calories_burned (CHECK >= 0)

EXERCISE
- PK: exercise_id
- exercise_name (UNIQUE, NOT NULL)
- muscle_group (NOT NULL)
- calories_per_unit (CHECK >= 0)

WORKOUT_EXERCISE
- PK: (workout_id, exercise_id, exercise_order)
- FK: workout_id → WORKOUT.workout_id (NOT NULL)
- FK: exercise_id → EXERCISE.exercise_id (NOT NULL)
- exercise_order (NOT NULL, CHECK > 0)
- sets (CHECK > 0)
- reps (CHECK > 0)
- weight (CHECK >= 0)
- duration_sec (CHECK > 0)

PROGRESS
- PK: progress_id
- FK: user_id → USER.user_id (NOT NULL)
- progress_date (NOT NULL)
- body_weight (CHECK > 0)
- body_fat_percent (CHECK >= 0 AND <= 100)
- notes
- UNIQUE(user_id, progress_date)

5. Reporting

Report 1: Workout Summary

```
mysql> SELECT
    ->     u.user_id,
    ->     u.email,
    ->     COUNT(DISTINCT m.meal_id)                AS total_meals,
    ->     COALESCE(SUM(m.total_calories), 0)        AS total_calories_consumed,
    ->     COUNT(DISTINCT w.workout_id)              AS total_workouts,
    ->     COALESCE(SUM(w.duration_min), 0)          AS total_workout_minutes
    -> FROM USER u
    -> LEFT JOIN MEAL    m ON m.user_id = u.user_id
    -> LEFT JOIN WORKOUT w ON w.user_id = u.user_id
    -> GROUP BY u.user_id, u.email
    -> ORDER BY total_calories_consumed DESC;
```

user_id	email	total_meals	total_calories_consumed	total_workouts	total_workout_minutes
92	user@gmail.com	1	400	1	23
96	test@gmail.com	1	96	1	50
67	user1@apples.com	0	0	0	0
97	bob123@gmail.com	0	0	1	45
71	user5@example.com	0	0	0	0
72	user6@example.com	0	0	0	0
73	user7@example.com	0	0	0	0
74	user8@example.com	0	0	0	0
75	user9@example.com	0	0	0	0
76	user10@example.com	0	0	0	0
77	user11@example.com	0	0	0	0
78	user12@example.com	0	0	0	0
79	user13@example.com	0	0	0	0
80	user14@example.com	0	0	0	0
81	user15@example.com	0	0	0	0
82	user16@example.com	0	0	0	0
83	user17@example.com	0	0	0	0
84	user18@example.com	0	0	0	0
85	user19@example.com	0	0	0	0
87	apples.com@.com	0	0	0	0
88	bob@gmail.com	0	0	0	0
89	4604@vt.edu	0	0	0	0
90	cs4604@vt.edu	0	0	0	0
91	jack@gmail.com	0	0	0	0
69	user3@example.com	0	0	0	0
93	admin@gmail.com	0	0	0	0
94	john@gmail.com	0	0	0	0
95	bob1@gmail.com	0	0	2	90
68	user2@example.com	0	0	0	0
70	user4@example.com	0	0	0	0

[8] - Result of the report

Explanation**:**

This report displays a summary of each user's activity, including:

Total number of meals logged

Total calories consumed

Total number of workouts

Total workout duration

It uses JOIN operations between the USER, MEAL, and WORKOUT tables and aggregates the data using COUNT and SUM.

Usefulness:

This report is useful because it gives users a high-level overview of their activity and consistency. It allows users to quickly understand how active they are and how much they are eating, which can help them adjust their fitness and diet habits.

Report 2: Weight Progress

```
mysql> SELECT
    ->     u.user_id,
    ->     u.email,
    ->     p.body_weight,
    ->     p.progress_date,
    ->     m.meal_name,
    ->     ROUND(AVG(m.total_calories), 1)  AS avg_calories_for_meal_type,
    ->     COUNT(m.meal_id)                 AS times_eaten
    -> FROM USER u
    -> JOIN PROGRESS p ON p.user_id = u.user_id
    -> JOIN MEAL     m ON m.user_id = u.user_id
    -> GROUP BY u.user_id, u.email, p.body_weight, p.progress_date, m.meal_name
    -> ORDER BY p.body_weight DESC, m.meal_name;
```

user_id	email	body_weight	progress_date	meal_name	avg_calories_for_meal_type	times_eaten
92	user@gmail.com	300	2026-04-24	breakfast	400	1
96	test@gmail.com	100.4	2026-04-22	Apple dinner	96	1

[9] - result of the second report

Explanation:

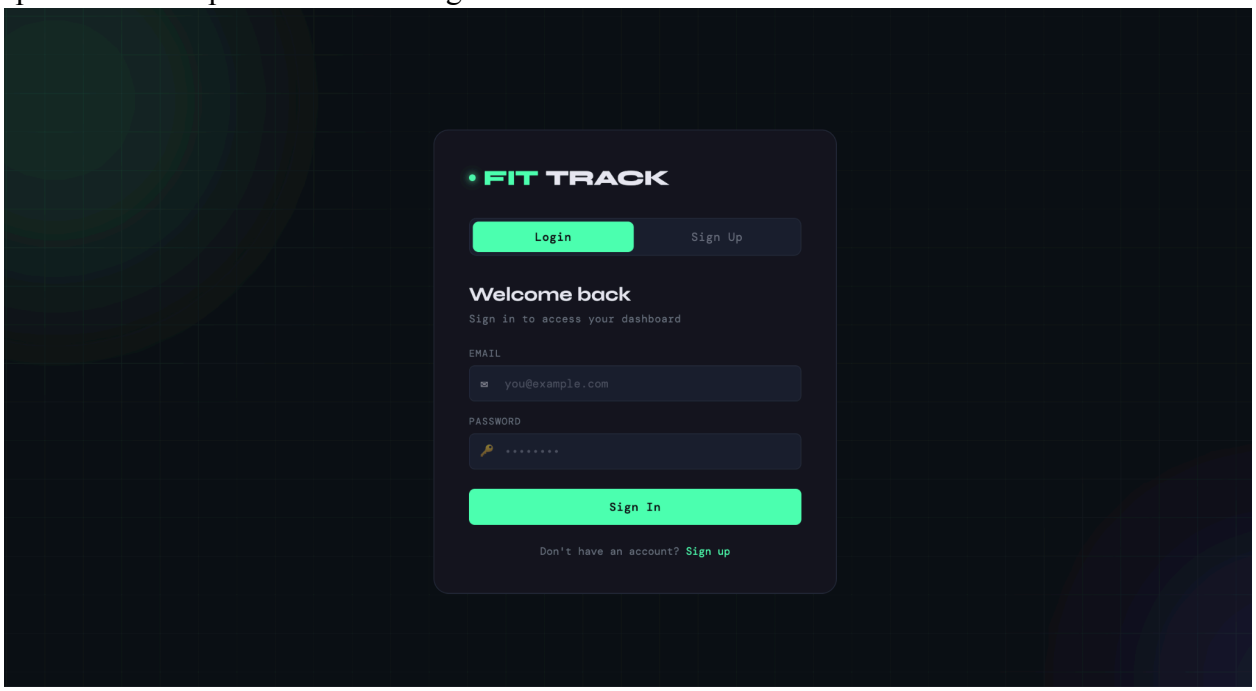
We use a join to make sure combine the meal data and progress data, then we order it in decreasing order of body weight.

Usefulness:
This report is helpful because it allows us to see a user, their meals, and their progress. Combining that data is very meaningful because it allows us to see how someone's meals may affect their progress. Additionally, we can see how often someone logs progress updates.

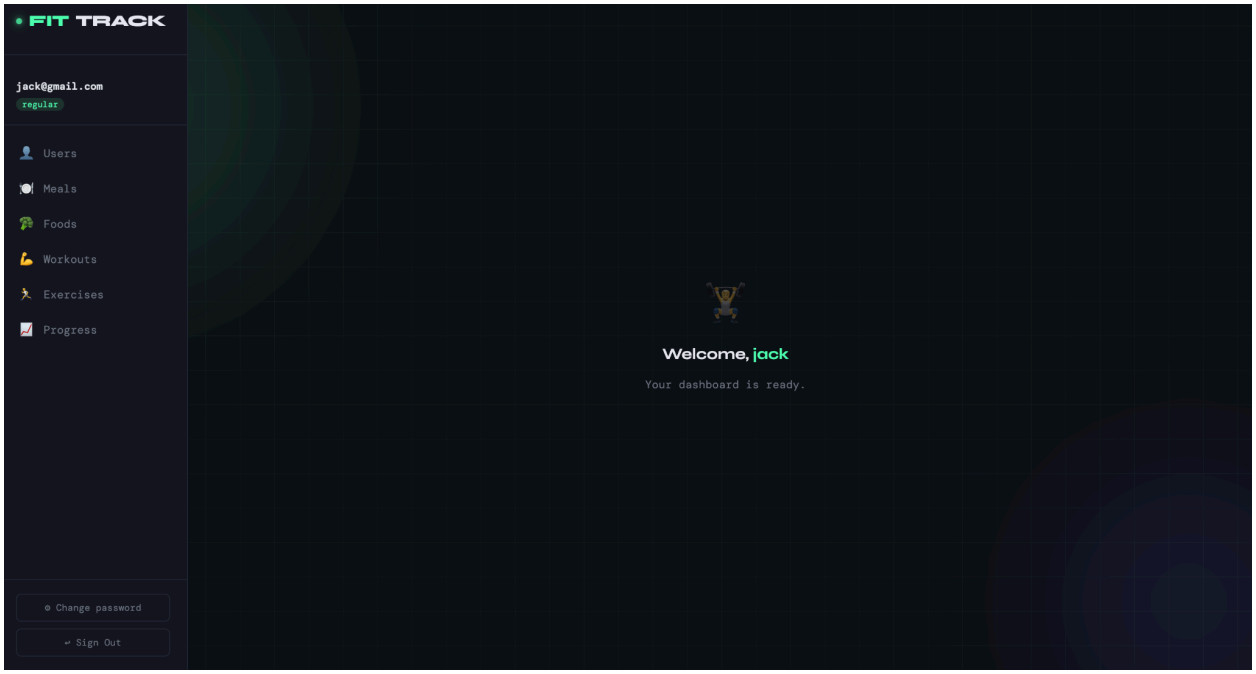
6. GUI / Interface

Describe and demonstrate your user interface.

Our interface is modern and easy to use without a technical background. Currently, we have the signup and login page completed. Once logged in, users can view their dashboard, featuring their data on the left hand side. After clicking on the left side of the intended feature, they can view their data, update it, delete it, or add new data. In the bottom left they also have the option to update their old password and to sign out.



[10] - interface showing the login/signup page



[11] - Screenshot showing the dashboard after a login

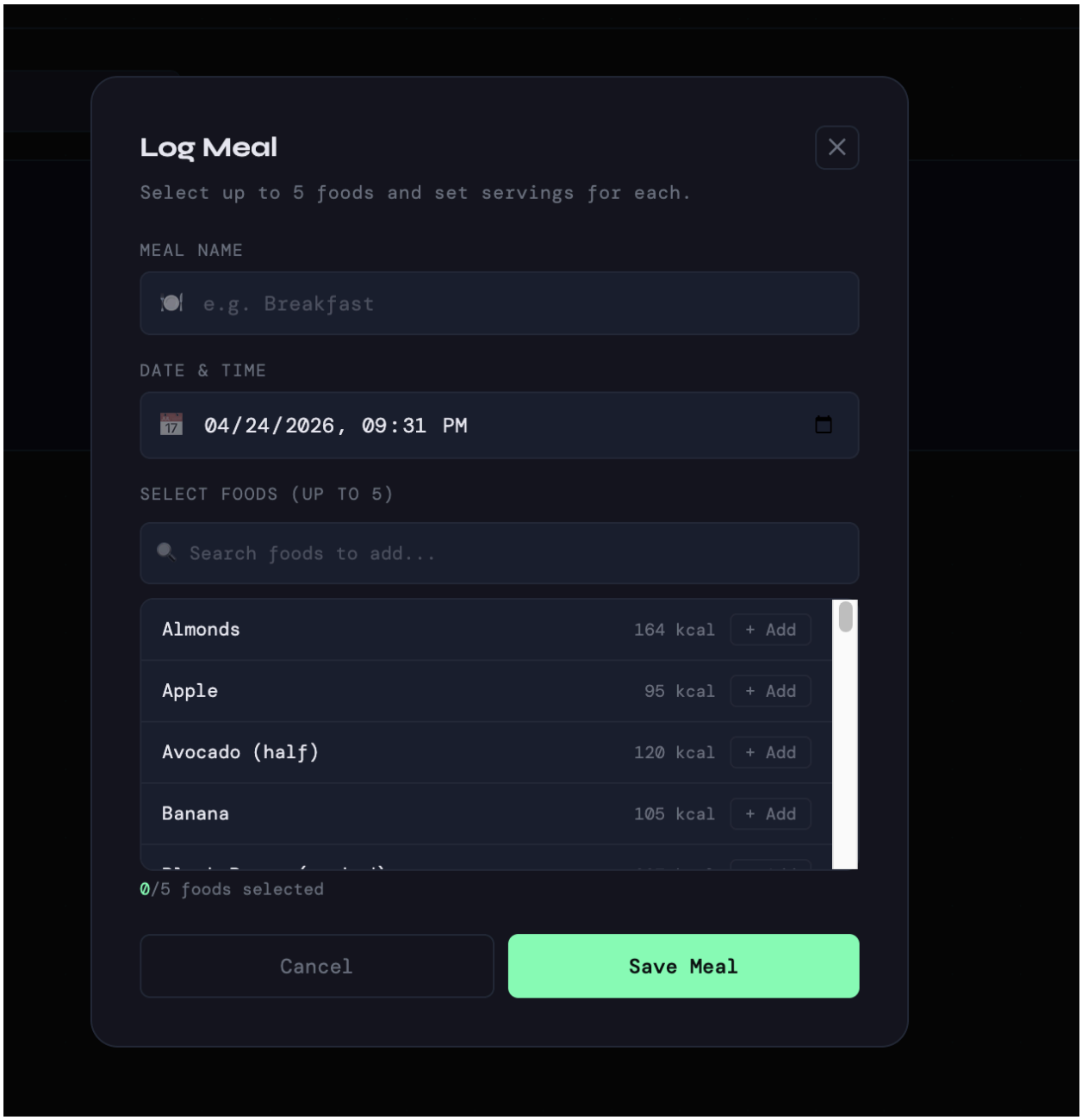
6.1 CRUD Operations

Your interface must support the following operations:

- Create
- Read
- Update
- Delete

Include before and after screenshots for each operation with brief explanations.

Create:



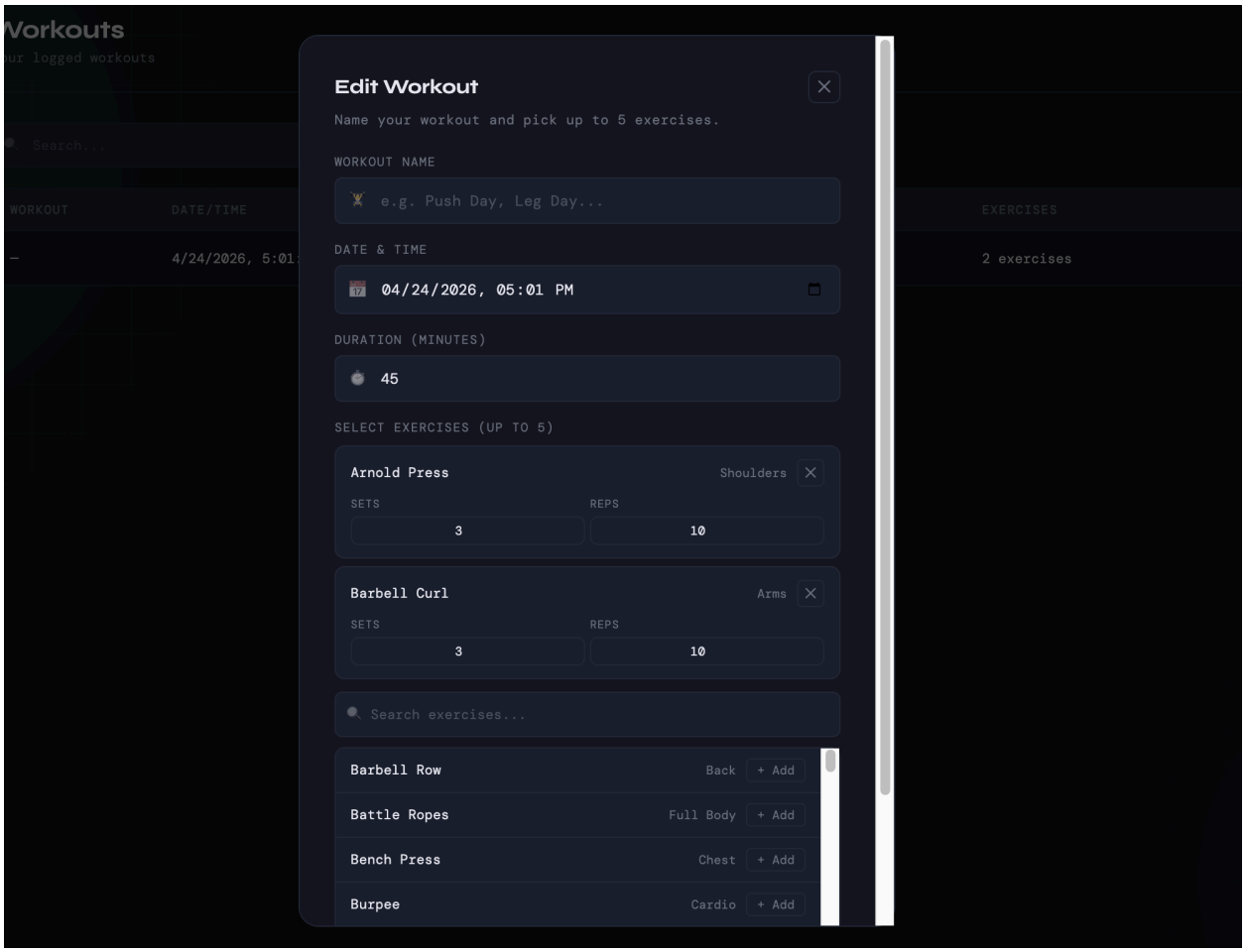
[12] - Create meal page

Read:

EXERCISE	MUSCLE GROUP	CAL/UNIT
Arnold Press	Shoulders	7
Barbell Curl	Arms	5
Barbell Row	Back	9
Battle Ropes	Full Body	10
Bench Press	Chest	8
Burpee	Cardio	10
Cable Crunch	Core	4
Cable Fly	Chest	6
Cable Kickback	Glutes	4
Clean and Press	Full Body	11
Crunch	Core	3
Cycling	Cardio	8

[13] - read exercises example

Update:



[14] - Update workout example

Delete:

WORKOUT	DATE/TIME	DURATION	CALORIES BURNED	EXERCISES	ACTIONS
–	4/24/2026, 5:01:00 PM	45 min	36 kcal	2 exercises	Edit Delete

[15] - Delete functionality for a workout

Before and After doing CRUD operations:

WORKOUT	DATE/TIME	DURATION	CALORIES BURNED	EXERCISES	ACTIONS
–	4/24/2026, 5:34:00 PM	12 min	63 kcal	3 exercises	Edit Delete
–	4/24/2026, 5:34:00 PM	120 min	378 kcal	5 exercises	Edit Delete
–	4/24/2026, 5:01:00 PM	45 min	36 kcal	2 exercises	Edit Delete

WORKOUT	DATE/TIME	DURATION	CALORIES BURNED	EXERCISES	ACTIONS
–	4/24/2026, 5:36:00 PM	230 min	21 kcal	1 exercise	Edit Delete
–	4/24/2026, 5:01:00 PM	45 min	36 kcal	2 exercises	Edit Delete
–	4/24/2026, 1:34:00 PM	100 min	363 kcal	4 exercises	Edit Delete

[17] [18] - Before and after of doing crud operations on the workouts page

7. Reflection

7.1 Individual Contributions

- Justin Hwang - I contributed primarily to the frontend development and overall system design. I worked on creating a clean and interactive user interface and ensured that the frontend properly communicated with the backend. I also helped refine the database schema and made sure that entity relationships were correctly structured.

- Landon Duba - I took the role of a full stack engineer. I contributed to working on the connection between the frontend, backend, and database. This included making endpoints and learning more about CORS, and flask. Additionally, I helped in other areas when needed, such as helping with frontend design or making changes to the database structure.

- Sid Madireddy - I contributed a little to the frontend design, as well as the database design aspects for the project. I also helped as a project management role making sure everyone was on the same page, things were done smoothly and on time. Additionally, I worked on testing the system to ensure that user actions through the interface correctly interacted with the backend and database.

- Vihaan Jaiswal - I contributed where needed across the project, mainly by assisting with frontend development, testing features, and helping connect different parts of the system. I supported the team by giving feedback on the user interface, checking that features worked correctly, and helping troubleshoot issues during development. Additionally, I stepped in to assist teammates when needed.

7.2 Team Reflection

One feature we are proud of:

One feature our team is proud of is allowing there to be admin users. While it was initially something we thought would be easy to implement, it turned out to be trickier than expected. For the admin user, we had several challenges. First, the admin had to have some way to sign up that was different from regular users. After evaluating different approaches, we decided that the admin must know a second additional password, and if they entered that correctly they were admitted to make an admin account. Another challenge was creating different views for the admin. The regular user doesn't have as many privileges, or can see as much data. So, when creating the admin we had to change the UI to grant them more permissions and to display the extra data. Ultimately, this required us to make frontend changes, backend changes, and lots of thinking about implementation choices.

Challenge and Future Changes:

One challenge we came across was setting up the database. While we went over our schema and were confident in it, actually creating and implementing it was new to us. So, there was a slight learning curve in the actual hands-on management of it. For example, we had to download MySQL, but at first we didn’t know for sure if that was what we wanted to use or if we wanted to go with another database. Then, we had to get familiar with the most efficient ways to implement things. For example, to populate the database, instead of adding examples one by one, we created a script to add many at once.

In the future, we would like to add AI features to the project. For example, the AI could look at the trends or possibly take in goals from a user and use that to recommend different workouts. Also, if we had more time another idea would be to try out different UI designs. We decided to go with a modern design to be easy to use, but other designs may have suited the application too.

8. Appendix

- **Link:** GitHub/GitLab Repository: https://github.com/landonDuba/4604-Database-Project